

For your architecture, you should absolutely write the harness code **inside the UI application itself** (as a distinct background service/module) and keep it completely separate from the models, runtimes, and backends. [1]

Because your APK is acting as the central orchestration hub, the UI and the harness belong in the same package, while the external components plug into it.

How to Structure This in Your APK

Think of your Kotlin APK as having two internal layers: the **UI Layer** (the steering wheel) and the **Harness Layer** (the engine block).

1. The Core Harness (An Internal Kotlin Service)

This runs in a background thread or a Foreground Service within your APK. It manages the loop. It holds the active state of the chat, formats the prompts, listens for the Model Context Protocol (MCP) tool-call JSON, and routes traffic. [2]

2. The Model & Backend Layer (Completely Externalized)

Whether you are hitting a local model running on-device via ONNX/llama.cpp, a fallback cloud API, or an external custom backend, the harness just treats them as an endpoint.

- Your harness code doesn't care *how* the model works.
- It just sends a standardized payload to whatever backend URL or socket the user selected in your UI.

3. The Runtime & Tool Layer (Connected via Sockets/Bridges)

Your WebViews and MCP tools are separate processes or isolated sandboxes. [3]

- When the harness parses an MCP tool call from the model, it sends a message across your socket to the tool/browser.
- The tool executes, and replies back to your harness over the socket. [4, 5]

Why Decoupling is Crucial for Your Specific Build

- **Seamless Fallbacks:** Because the harness is decoupled from the backend, when your local on-device model fails or runs out of VRAM, your harness logic can instantly redirect the exact same context window to your Cloud Front-end or fallback backend without interrupting the user interface.
- **Pluggable Runtimes:** Since you want to upload any runtime, your harness acts as an abstract traffic controller. It uses standard interfaces (like MCP or JSON-RPC over web sockets) so any new runtime or tool can register itself with your APK. [6]

- **UI Responsiveness:** Keeping the orchestration loop in a separate Kotlin module keeps your UI fluid. The UI merely observes the harness state and renders the incoming text tokens or tool animations.

How Data Moves Through Your APK

1. **User** types a prompt in your Kotlin UI.
2. **UI** passes the text to your internal **Harness Module**.
3. **Harness** grabs the active history, packages it, and sends it to the selected **Backend/Model** (Local or Cloud).
4. **Backend** responds: *"I need to search the web using an MCP tool."*
5. **Harness** intercepts this, pauses the model connection, and sends a socket event to your **WebView/MCP socket**.
6. **MCP Tool** executes, returns the data to the **Harness**.
7. **Harness** passes the tool result back to the **Backend**, gets the final answer, and updates the **UI**. [7, 8, 9, 10, 11]

Would you like to map out the **Kotlin interface structures** or the basic architecture of how your background harness service should talk to your UI components?

- [1] <https://www.cs.nmsu.edu>
- [2] <https://visualstudiomagazine.com>
- [3] <https://medium.com>
- [4] <https://ai.plainenglish.io>
- [5] <https://medium.com>
- [6] <https://pub.towardsai.net>
- [7] <https://khemlall-mangal.medium.com>
- [8] <https://dev.to>
- [9] <https://nayakplaban.medium.com>
- [10] <https://boringbot.substack.com>
- [11] <https://stevekinney.com>